BCA SEMESTER -4
Programming With C Sharp
April 2023

Q 1 (A) Attempt the following : **4**

1. Full form of CLR . - **Common Language Runtime**
2. Array of array is called _____ array.- **Jagged**
3. _____ keyword is used to include namespaces. - **Using**
4. Full form of FCL. - **Framework Class Library**

(B) Attempt Any One: **2**

1. Explain Foreach loop with example.

the `foreach` loop simplifies iterating over collections like arrays or lists by eliminating the need for explicit indexing. Its syntax is:

```
foreach (var item in collection)
{
   // Code to execute for each item
}
```

For example, to display all elements in an array:

```
using System;
class loopworks
{
  static public void Main()
   {
      Console.WriteLine("Print array:");   //displays text on output
window
      int[] arr_array = new int[] { 0, 1, 2, 3, 4, 5, };    // array
creation
       // foreach loop begins and runs till last item
      foreach(int i in arr_array)
      {
         Console.Write(i);
      }
   }
} // end of code
Output : 0 1 2 3 4 5
```

2. List types of project in IDE.

In Visual Studio, C# projects can be created for various application types, including:

1. **Console Application**: Command-line programs suitable for utilities and scripts.
2. **Windows Forms Application**: Desktop applications with a graphical user interface using Windows Forms.
3. **WPF Application**: Desktop applications with advanced UI capabilities using Windows Presentation Foundation.
4. **ASP.NET Core Web Application**: Cross-platform web applications and services using ASP.NET Core.
5. **Blazor App**: Interactive web UIs using C# instead of JavaScript.
6. **Class Library**: Reusable assemblies containing classes and methods for other projects.
7. **Unit Test Project**: Projects for writing and running unit tests using frameworks like MSTest or xUnit.
8. **Worker Service**: Background services for long-running tasks.
9. **Azure Functions**: Serverless compute services for event-driven code.

10. **.NET MAUI App**: Cross-platform applications for Android, iOS, macOS, and Windows.

(C) Attempt any one: **3**

1. <u>What is Boxing and Unboxing?</u>

In C#, **boxing** and **unboxing** are processes that enable the conversion between value types (e.g., int, double) and reference types (object). These operations are fundamental to the type system in C#.

Boxing:

This is the process of converting a value type to a reference type by encapsulating the value inside an object or an interface it implements. When boxing occurs, the value is wrapped within a new object and stored on the managed heap. Boxing is an implicit operation.

***Example*:**

```
int number = 123;
object boxedNumber = number; // Boxing
```

In this example, the integer number is boxed into the object boxedNumber.

Unboxing:

This is the process of extracting the value type from the object. It requires an explicit cast to convert the object back to the original value type. Unboxing involves checking the object instance to ensure it holds a boxed value of the specified value type and then copying the value from the heap to the stack.

***Example*:**

```
object boxedNumber = 123;
int number = (int)boxedNumber; // Unboxing
```

Here, the object boxedNumber is explicitly cast back to an integer.

2. <u>Explain if statement with syntax and example.</u>

the `if` statement is a fundamental control structure that allows you to execute a block of code based on whether a specified condition evaluates to `true`. This enables conditional execution, where different code paths can be taken depending on varying conditions.

Syntax:

```
if (condition)
{
   // Code to execute if the condition is true
}
```

condition: A Boolean expression that evaluates to true or false.

Example:

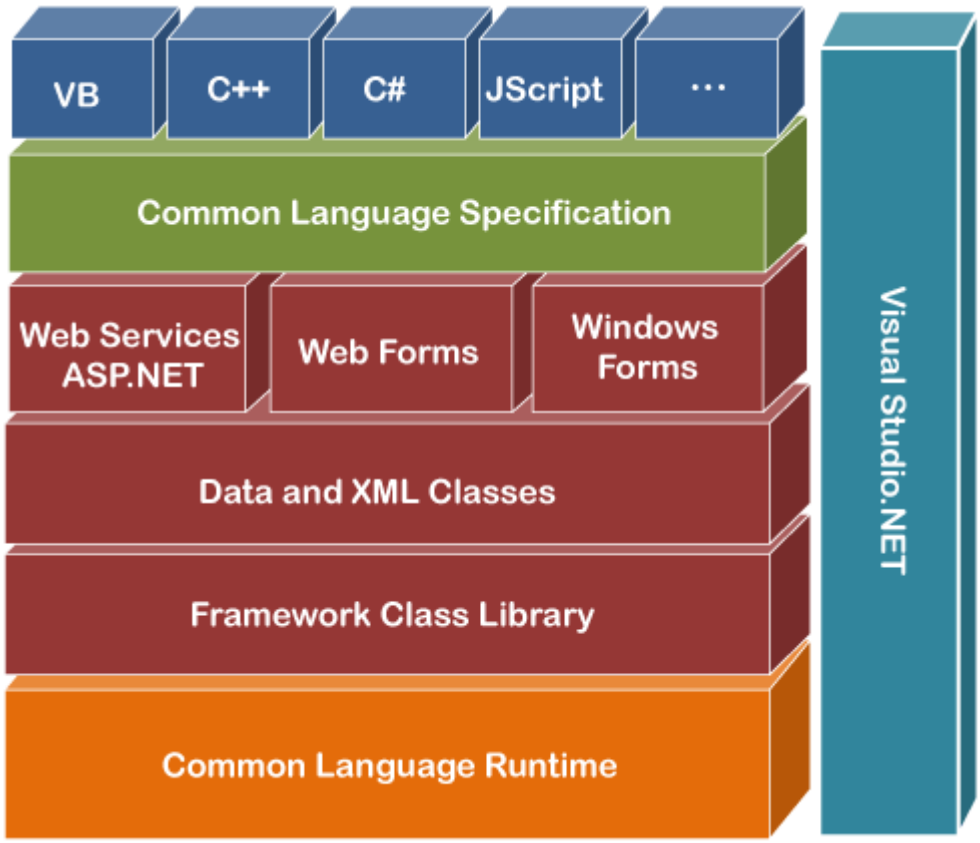
```
using System;
class Program
{
   static void Main()
   {
      int number = 10;

      if (number > 5)
      {
         Console.WriteLine("The number is greater than 5.");
      }
   }
}
Output: The number is greater than 5.
```

In this example, the condition number > 5 evaluates to true since number is 10. Therefore, the message "The number is greater than 5." is printed to the console.

(D) Attempt any One: **5**

1. Explain Architecture of .Net framework.

The .NET Framework is a comprehensive and consistent programming model developed by Microsoft for building applications with visually stunning user experiences, seamless and secure communication, and the ability to model a range of business processes. Its architecture is designed to provide a managed execution environment, simplified development, and integration with a variety of programming languages.



Key Components of the .NET Framework Architecture:

1. **Common Language Runtime (CLR):**
   - The CLR is the execution engine for .NET applications. It provides services such as memory management, type safety, exception handling, garbage collection, security, and thread management.
   - Code compiled for the .NET Framework is converted into an intermediate language called Common Intermediate Language (CIL). During execution, the CLR's Just-In-Time (JIT) compiler translates this CIL code into native machine code specific to the underlying architecture.
2. **.NET Framework Class Library (FCL):**
   - The FCL is a comprehensive collection of reusable classes, interfaces, and value types that provide access to system functionality. It is organized into a hierarchy of namespaces and includes classes for tasks such as file reading and writing, graphic rendering, database interaction, and XML document manipulation.
   - The FCL extends the Base Class Library (BCL) by adding libraries for specific functionalities like Windows Forms, ASP.NET, and Windows Presentation Foundation (WPF).
3. **Common Language Infrastructure (CLI):**

- The CLI provides a language-neutral platform for application development and execution. By implementing core aspects of the .NET Framework within the scope of the CLI, these functions are available across multiple programming languages supported by the framework.

4. **Assemblies:**
    - Assemblies are the building blocks of .NET applications. They are compiled code libraries used for deployment, versioning, and security. An assembly can consist of one or more files, and it contains a manifest that provides metadata about the assembly's contents.

2. <u>Explain Data types in details</u>.

C#, data types define the kind of data that variables can store, ensuring type safety and efficient memory usage. They are broadly categorized into **Value Types** and **Reference Types**.

1. Value Types:

Value types directly contain their data, and each variable has its own copy of the data. They are typically stored on the stack. Assigning one value type variable to another copies the actual data. Common value types include:

- **Integral Types**:
    - int (System.Int32): Represents 32-bit signed integers, ranging from -2,147,483,648 to 2,147,483,647.
    - byte (System.Byte): Represents 8-bit unsigned integers, ranging from 0 to 255.
- char (System.Char): Represents a single 16-bit Unicode character.
- **Floating-Point Types**:
    - float (System.Single): Represents a single-precision 32-bit floating point, suitable for storing 7-digit precision numbers.
    - double (System.Double): Represents a double-precision 64-bit floating point, accommodating 15-16 digit precision.
- **Other Types**:
    - bool (System.Boolean): Represents Boolean values, true or false.
    - decimal (System.Decimal): A 128-bit data type suitable for financial and monetary calculations requiring high precision.
    - enum: Defines a set of named integral constants.
    - struct: A composite value type that can encapsulate related variables.

2. Reference Types:

Reference types store references to their data (objects), meaning variables point to the memory location where the data is stored. They are typically allocated on the heap. Assigning one reference type variable to another copies the reference, not the actual data, so changes via one variable affect the same object accessed by the other. Common reference types include:

- **object (System.Object)**: The base type from which all other types derive.
- **string (System.String)**: Represents a sequence of Unicode characters.
- **Arrays**: Collections of elements of the same type.
- **Classes and Interfaces**: Blueprints for creating objects and defining object behaviors.

Understanding the distinction between value and reference types is crucial for effective memory management and application performance. For instance, modifying a reference type variable can impact all references to the same object, whereas value type variables operate independently.

Q 2 (A) Attempt the following : 4

1. _____ class will prevent inheritance. - **Sealed**
2. _____ method is used to add value to the end of queue. - **Enqueue**
3. We can use memory through pointer using ___ keyword. - **unsafe**
4. Which operator is used to add another delegate to multicast delegate? - **+= operator**

(B) Attempt any One : **2**

1. <u>Give the syntax for to define sealed class</u>.

To define a sealed class—one that cannot be inherited—you use the sealed keyword before the class keyword. The **syntax** is as follows

```
sealed class ClassName
{
   // Class members go here
}
```

Example:

```
sealed class Animal
{
   public void MakeSound()
   {
      Console.WriteLine("Animal makes a sound.");
   }
}
```

In this example, Animal is a sealed class, meaning no other class can inherit from it. Attempting to do so will result in a compile-time error.

2. <u>What is Method Overloading?</u>

Method overloading allows multiple methods within the same class to share the same name but differ in their parameter lists—either by number, type, or order of parameters. This feature enhances code readability and usability by enabling methods to handle different types or numbers of inputs while maintaining a consistent method name.

Example:

```
public class Display
{
   public void Show()
   {
      Console.WriteLine("No parameters");
   }

   public void Show(string message)
   {
      Console.WriteLine("Message: " + message);
   }
}
```

In this example, the Show method is overloaded to handle both zero parameters and a single string parameter. Method overloading improves code maintainability and provides flexibility in method usage.

(C) Attempt Any One : **3**

1. <u>What is Delegate? Give an Example</u>.

In C#, a **delegate** is a type that defines a method signature, allowing methods to be passed as parameters. Delegates enable the creation of references to methods with specific parameter lists and return types, facilitating callback methods and event handling.

Defining a Delegate:

To declare a delegate, use the `delegate` keyword followed by the method signature:

public delegate void Notify(string message);

This declaration defines a delegate named `Notify` that can reference any method with a `void` return type and a single `string` parameter

Instantiating and Using a Delegate:
Once a delegate is defined, you can instantiate it and assign a method that matches its signature:

```
public class Process
{
   public static void DisplayMessage(string msg)
   {
      Console.WriteLine(msg);
   }
}
class Program
{
   static void Main()
   {
      Notify notifier = new Notify(Process.DisplayMessage);
      notifier("Hello, World!");
   }
}
```

2. <u>Differentiate: Ref v/s Out.</u>

Both ref and out keywords enable passing arguments by reference, allowing methods to modify the caller's variables. However, they serve different purposes and have distinct requirements:

- **Initialization**:
  - **ref**: The variable must be initialized before being passed to the method. This allows the method to read and modify its value.
  - **out**: The variable does not need prior initialization. The method is responsible for assigning it a value before returning.
- **Usage Intent**:
  - **ref**: Use when the method needs to both read and modify the existing value of the parameter.
  - **out**: Use when the method's primary purpose is to assign a new value to the parameter, often to return multiple results.

(D) Attempt Any One : **5**

1. <u>What is Collection? Explain any one collection class in details</u>

a **collection** is a class designed to manage groups of related objects, providing functionalities for storage, retrieval, and manipulation. Unlike arrays with fixed sizes, collections offer dynamic sizing, allowing elements to be added or removed as needed.

Types of Collections in C#:

1. **Generic Collections**: Located in the System.Collections.Generic namespace, these collections are type-safe at compile time, enforcing type constraints and preventing runtime errors related to type mismatches. Examples include List<T>, Dictionary<TKey, TValue>, and Queue<T>.
2. **Non-Generic Collections**: Found in the System.Collections namespace, these collections can store any object type, leading to potential boxing and unboxing overhead. Examples are ArrayList, Hashtable, and Queue.

ArrayList is one of the most flexible data structure from CSharp Collections. ArrayList contains a simple list of values. ArrayList implements the IList interface using an array and very

easily we can add, insert, delete, view etc. It is very flexible because we can add without any size information, which is it will grow dynamically and also shrink.

```
Add : Add an Item in an ArrayList
Insert : Insert an Item in a specified position in an ArrayList
Remove : Remove an Item from ArrayList
RemoveAt: remove an item from a specified position
Sort : Sort Items in an ArrayList
```

How to add an Item in an ArrayList ?

```
Syntax : ArrayList.add(object)
object : The Item to be add the ArrayList
ArrayList arr;
arr.Add("Item1");
```

How to Insert an Item in an ArrayList ?

```
Syntax : ArrayList.insert(index,object)
index : The position of the item in an ArrayList
object : The Item to be add the ArrayList
ArrayList arr;
arr.Insert(3, "Item3");
```

How to remove an item from arrayList ?

```
Syntax : ArrayList.Remove(object)
object : The Item to be add the ArrayList
arr.Remove("item2")
```

How to remove an item in a specified position from an ArrayList ?

```
Syntax : ArrayList.RemoveAt(index)
index : the position of an item to remove from an ArrayList
ItemList.RemoveAt(2)
```

How to sort ArrayList ?

```
Syntax : ArrayList.Sort()
```

An ArrayList can be constructed
Like this:

```
ArrayList list = new ArrayList();
```

We can also specify the initial Capacity of the ArrayList by passing an integer value to the constructor:

```
ArrayList list = new ArrayList(20);
```

PROGRAM:

```
static void Main()
{
        ArrayList list = new ArrayList();
        list.Add(45);
        list.Add(87);
        list.Add(12);
        foreach(int num in list)
       {
          Console.Write(num);
        }
}
```

Output :45 87 12

2. Explain single and Multicast Delegate with example.

DELEGATES:

A delegate in C# is similar to a function pointer in C or C++. Using a delegate allows the programmer to encapsulate a reference to a method inside a delegate object. The delegate object can then be passed to code which can call the referenced method, without having to know at compile time which method will be invoked.

When you want to create a delegate in C# you make use of delegate keyword.

The name of your delegate can be whatever you desire. However, you must define the delegate to match the signature of the method it will point to. For example the following delegate can point to any method taking two integers and returning an integer.

```
public delegate int DelegateName(int x, int y);
```

PROGRAM:

```
using System;
namespace Delegate
{
    public delegate int MyDelegate(int x,int y);
class MyClass
{
    public static int add(int x, int y)
    {
        return x + y;
    }
    public static int mul(int x, int y)
    {
        return x * y;
    }
}
class Program
 {
        static void Main(string[] args)
        {
            MyDelegate del1 = new MyDelegate(MyClass.add);
            int result = del1(5, 5);
            Console.WriteLine(" 5 + 5 ={0}", result);
            MyDelegate del2 = new MyDelegate(MyClass.mul);
            int resultm = del2(5, 5);
            Console.WriteLine(" 5 X 5 ={0}", resultm);
            Console.ReadKey();
        }
    }
}
```

Output:

5 + 5 = 10

5 X 5 = 25

MULTICAST DELEGATES:

Delegate's ability to multicast means that a delegate object can maintain a list of methods to call, rather than a single method if you want to add a method to the invocation list of a delegate object , you simply make use of the overloaded += operator, and if you want to remove a method from the invocation list you make use of the overloaded operator -= .

```
using System;
public delegate void MulticastDelegate(int x, int y);
public class MyClass
{
    public static void Add(int x, int y)
    {
        Console.WriteLine("You are in Add() Method");
        Console.WriteLine("{0} + {1} = {2}\n", x, y, x + y);
    }
    public static void Multiply(int x, int y)
    {
        Console.WriteLine("You are in Multiply() Method");
        Console.WriteLine("{0} X {1} = {2}", x, y, x * y);
    }
}
class Program
{
    static void Main(string[] args)
    {
        MulticastDelegate del = new MulticastDelegate(MyClass.Add);
        del += new MulticastDelegate(MyClass.Multiply);
        Console.WriteLine("calling Add() and Multiply() Methods.");
        del(5, 5);
        del -= new MulticastDelegate(MyClass.Add);
        Console.WriteLine("\n****Add() Method removed.****\n");
        del(5, 5);
    }
}
```

```
calling Add() and Multiply() Methods
You are in add() method
5 + 5 =10
You are in multiply() method
5 X 5 =25
****Add() Method removed.****
You are in multiply() method
5 X 5 =25
```

Q 3 (A) Attempt the following : **4**

1. Default value of interval property of timer is___ - **100 milliseconds.**
2. ___id default property of textbox control. - **Text property**
3. MDI stands for _____ - **Multiple Document Interface**
4._____is default event of timer control. - **Tick event**

(B) Attempt any one : **2**

1**. <u>Differentiate TextBox v/s RichTextBox.</u>**

Both TextBox and RichTextBox controls are used for text input, but they serve different purposes:

- **TextBox**: Ideal for plain text input, supporting single or multi-line modes. It consumes fewer system resources and supports data binding, making it suitable for simple text entry forms.
- **RichTextBox**: Designed for rich text content, it allows text formatting (e.g., bold, italics), embedding images, and handling complex documents. However, it does not natively support data binding and is more resource-intensive.

2. <u>List out property of Button control.</u>

The Button control in Windows Forms (System.Windows.Forms.Button) offers various properties to customize its appearance and behavior. Here are some key properties:

- **Text**: Sets the text displayed on the button.
- **Image**: Assigns an image to the button.
- **ImageAlign**: Determines the alignment of the image within the button.
- **TextAlign**: Sets the alignment of the text within the button.
- **FlatStyle**: Specifies the appearance style of the button (e.g., Flat, Popup, Standard, System).
- **DialogResult**: Defines the return value of a form when the button is clicked in a modal dialog.
- **Enabled**: Indicates whether the button is interactive.
- **Visible**: Determines the visibility of the button on the form.
- **BackColor**: Sets the background color of the button.
- **ForeColor**: Sets the foreground color of the button's text.
- **Font**: Defines the font style of the button's text.

(C) Attempt Any One: 3

1. <u>Explain SaveFileDialog with Example</u>.

The SaveFileDialog class from the System.Windows.Forms namespace provides a standard dialog box that prompts users to specify a file name and location for saving a file. This dialog simplifies file-saving operations by offering a familiar interface consistent with Windows applications.

Key Properties of SaveFileDialog:

- **Filter**: Determines the file types displayed and saved. For example, setting Filter = "Text files (*.txt)|*.txt|All files (*.*)|*.*"; allows users to choose between text files and all files.
- **FilterIndex**: Sets the default selected filter. It's a one-based index; for instance, setting FilterIndex = 1; selects the first filter as default.
- **DefaultExt**: Specifies the default file extension. If the user doesn't provide an extension, this value is appended. For example, DefaultExt = "txt"; ensures files are saved with a .txt extension if none is specified.
- **AddExtension**: Indicates whether the dialog box automatically adds an extension to a file name if the user omits it.

Example

```
private void SaveButton_Click(object sender, EventArgs e) {
   SaveFileDialog saveFileDialog1 = new SaveFileDialog();
   saveFileDialog1.InitialDirectory = @ "C:\";
   saveFileDialog1.Title = "Save text Files";
   saveFileDialog1.CheckFileExists = true;
   saveFileDialog1.CheckPathExists = true;
   saveFileDialog1.DefaultExt = "txt";
   saveFileDialog1.Filter = "Text files (*.txt)|*.txt|All files (*.*)|*.*";
   saveFileDialog1.FilterIndex = 2;
   saveFileDialog1.RestoreDirectory = true;
   if (saveFileDialog1.ShowDialog() == DialogResult.OK) {
      textBox1.Text = saveFileDialog1.FileName;
   }
}
```

2. <u>Explain Picture Box with Example.</u>

In C#, the **PictureBox** control is a Windows Forms component designed to display images in various formats, such as BMP, JPEG, GIF, PNG, and others. It provides a straightforward way to incorporate images into your application's user interface.

Key Properties of PictureBox:

- **Image:** Sets or retrieves the image displayed in the PictureBox.
  - **ImageLocation:** Specifies the path or URL of the image to display.
  - **SizeMode:** Determines how the image is displayed within the PictureBox. Options include
  - Normal: The image is placed in the upper-left corner of the PictureBox.
  - StretchImage: The image is stretched or shrunk to fit the PictureBox
  - AutoSize: The PictureBox adjusts its size to match the image's size.
  - CenterImage: The image is centered within the PictureBox.
  - Zoom: The image is proportionally resized to fit the PictureBox without distortion.
- **BorderStyle:** Sets the border style of the PictureBox, such as None, FixedSingle, or Fixed3D.

Q 4 (A)Attempt the following : 4

1. ADO stands for____ - **ActiveX Data Objects**
2. DataReader is a Disconnected Architecture component.(True/False) - **False**
3. To Connect with SqlServer _____ namesapce - **System.Data.SqlClient**
4. Fill method is used with DataReader.(True/False) - **False**

(B)Attempt Any One : 2

<u>1. Explain Command Object.</u>

A **Command Object** is used to execute **SQL queries**, **stored procedures**, or **commands** against a data source (like a SQL Server database). It is responsible for:

- Executing SELECT, INSERT, UPDATE, DELETE statements.
- Calling stored procedures.
- Performing scalar queries (returning a single value).
- Executing queries that return data (via a DataReader).
- Executing non-query commands (e.g., updating data).

<u>2. What is DataTable?</u>

a **DataTable** is an in-memory representation of a single table of data, part of the **System.Data** namespace in ADO.NET. It consists of rows and columns, similar to a database table. You can define its schema, add data manually, or fill it using a DataAdapter from a database query. DataTable supports features like filtering, sorting, and data relations. It's often used for temporary storage, data manipulation, or binding to UI controls like grids. DataTables can also be part of a DataSet, allowing more complex data handling scenarios in disconnected environments.

(C)Attempt Any One : 3

1. **<u>Explain DataReader with example.</u>**

A **DataReader** is used to read data from a database in a **forward-only**, **read-only** manner. It provides fast, efficient access to data, especially useful for reading large amounts of data quickly. DataReader is part of ADO.NET and is accessed through classes like SqlDataReader for SQL Server.

Unlike DataTable, a DataReader doesn't store data in memory—it's connected and reads one row at a time, making it ideal for performance-sensitive scenarios.

```
using System;
using System.Data.SqlClient;

string connectionString = "your_connection_string";
string query = "SELECT Name, Age FROM Employees";

using (SqlConnection connection = new SqlConnection(connectionString))
{
    SqlCommand command = new SqlCommand(query, connection);
    connection.Open();
    SqlDataReader reader = command.ExecuteReader();

    while (reader.Read())
    {
        Console.WriteLine($"Name: {reader["Name"]}, Age: {reader["Age"]}");
    }

    reader.Close();
}
```

2. **Differentiate between ExecuteNonQuery() and ExecuteScalar().**

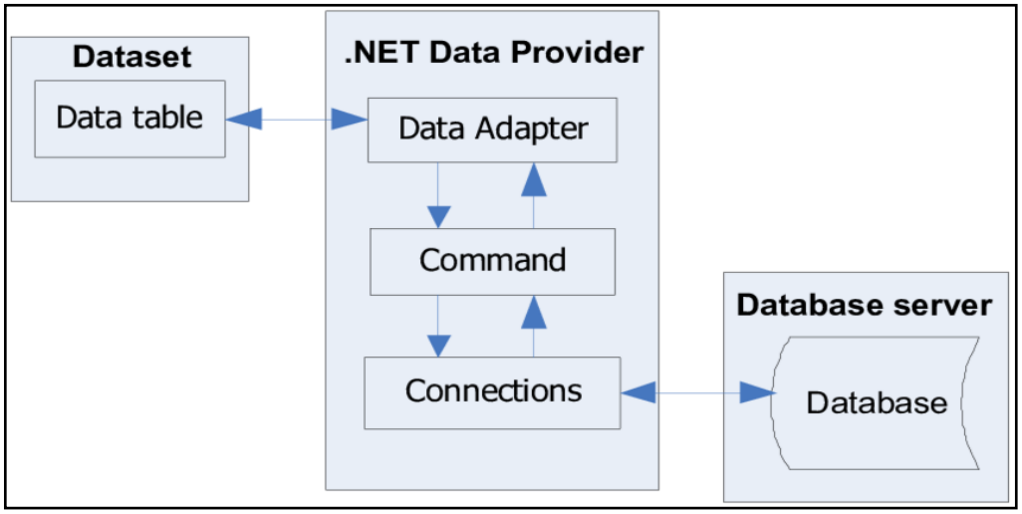
Here's a clear comparison between **ExecuteNonQuery()** and **ExecuteScalar()** in C#:

Feature	ExecuteNonQuery()	ExecuteScalar()
Purpose	Executes a command that **does not return rows**	Executes a command that **returns a single value**
Common Use	INSERT, UPDATE, DELETE, or DDL statements	SELECT statements that return a **single value**
Return Type	int (number of rows affected)	object (first column of the first row)
Returns Data?	No	Yes (a single value)
Performance	Very fast for write operations	Fast for retrieving a single value

(D) Attempt Any One **5**

1. Explain the Architecture of ADO.NET.

ADO.NET (ActiveX Data Objects .NET) is a set of classes in the .NET Framework that allows applications to **connect to data sources**, **retrieve**, **manipulate**, and **update** data — typically from relational databases like SQL Server, MySQL, Oracle, etc.



ADO.NET Architecture is a part of the .NET Framework designed to handle data access from various sources such as SQL Server, Oracle, and XML. It provides a consistent and efficient way to connect, retrieve, manipulate, and update data. The architecture is divided into two main models: **Connected** and **Disconnected**.

Connected Architecture

This model maintains a continuous connection with the data source during operations. It is best suited for real-time, read-only access to large amounts of data.

Key components:

- **SqlConnection**: Establishes and manages a connection to the database.
- **SqlCommand**: Executes SQL queries or stored procedures.
- **SqlDataReader**: Provides fast, forward-only, read-only access to data.
- **SqlTransaction**: Manages transactions for executing multiple operations atomically.

Disconnected Architecture

This model retrieves data from the database, stores it in memory, and closes the connection, allowing manipulation of data offline. It's suitable for multi-tier or distributed applications.

Key components:

- **DataAdapter**: Acts as a bridge between the database and in-memory data (DataSet/DataTable). It handles data retrieval (Fill) and updates (Update).
- **DataSet**: An in-memory cache that can store multiple DataTable objects and maintain relationships between them.
- **DataTable**: Represents a single table of in-memory data.

2. Expain Connection and DataAdapter Object in Details.

Connection Object

Purpose:

The **Connection Object** (e.g., SqlConnection) is used to establish a **link between a C# application and a data source** like SQL Server.

Key Class:

SqlConnection (for SQL Server), from System.Data.SqlClient.

Important Properties:

- ConnectionString: Contains details like server name, database, user credentials.
- State: Indicates the current state (Open, Closed, etc.).

Important Methods:

- Open(): Opens the connection to the database.
- Close(): Closes the connection.

Example:

```
string connStr = "Server=.;Database=MyDB;Trusted_Connection=True;";
SqlConnection conn = new SqlConnection(connStr);
conn.Open();
// Use the connection
conn.Close();
```

DataAdapter Object

Purpose:

The **DataAdapter** acts as a **bridge between a DataSet/DataTable and the database**. It is used to **fill data into a DataSet** and **update the database** using disconnected architecture.

Key Class:

SqlDataAdapter (for SQL Server), from System.Data.SqlClient.

Main Uses:

- Populate a DataTable or DataSet from a query.
- Automatically generate INSERT, UPDATE, DELETE commands with SqlCommandBuilder.
- Push changes made in DataTable back to the database.

Important Methods:

- Fill(): Fills a DataTable/DataSet with query results.
- Update(): Updates the database with changes made in the DataTable/DataSet.

Example:

```
string connStr = "Server=.;Database=MyDB;Trusted_Connection=True;";
string query = "SELECT * FROM Employees";
SqlConnection conn = new SqlConnection(connStr);
SqlDataAdapter adapter = new SqlDataAdapter(query, conn);
DataTable dt = new DataTable();
adapter.Fill(dt);
// Modify dt here...
// Optional: Update the database
// SqlCommandBuilder builder = new SqlCommandBuilder(adapter);
// adapter.Update(dt);
```

Q 5(A) Attempt the following. **4**

1. _____control is used on form to view the crystal Report. - **CrystalReportViewer**
2. By default, how many section are provided in crystal Report - **Five sections**
3._____ field of crystal report is used to create Formula. -**Formula Field**
4. Drop Down is a type of Crystal Report.(True/False) - **False.**

(B)Attempt Any One : **2**

1. List out types of Reports.

Here are the main types of reports in Crystal Reports:

1. **Standard Report** – Displays data in a simple tabular or list format.
2. **Subreport** – A report embedded within another, showing related data.
3. **Cross-Tab Report** – Summarizes data in a matrix format (rows and columns).
4. **Chart Report** – Graphically represents data (e.g., bar, line, or pie charts).
5. **Drill-Down Report** – Allows users to click on summarized data for more detailed information.
6. **Parameterized Report** – Accepts user input to filter data.
7. **Mailing Label Report** – Used to create formatted labels.
8. **Form Report** – Creates printable forms like invoices and receipts.

2. Write the name of any four Special Fields of Crystal Report.

Here is a list of all the **special fields** in Crystal Reports:

1. **Record Number**
   - Displays the number of the current record in the report.
2. **Page Number**
   - Displays the current page number of the report.
3. **Total Page Count**
   - Displays the total number of pages in the report.
4. **Report Title**
   - Displays the title of the report (set in the Report Header).
5. **Date/Time**
   - Displays the current date and time when the report is being viewed or printed.
6. **User Name**
   - Displays the name of the user who is viewing or printing the report.

(C) Attempt Any One: 3

1. <u>Explain types of setup Projects.</u>

setup projects are used to create an installation package for deploying applications. The setup project packages all necessary files, registries, and dependencies into a single executable to easily install the application on other systems. Here are the common types of setup projects:

1. **Windows Application Setup**: Used to create an installer for Windows Forms or WPF applications. It includes the application's executable, required libraries, and dependencies.
2. **Web Application Setup**: Deploys web applications (ASP.NET) by including web files, configurations, and settings needed for hosting on a server.
3. **Class Library Setup**: For packaging a class library (DLL) and its dependencies, making it easy to install and use in other projects.
4. **Merge Module Setup**: A modular approach that allows you to create reusable components (like shared libraries or controls) for distribution.
5. **Custom Setup Project**: Allows you to define custom installation steps, like checking system requirements or configuring user options.

2. <u>Explain summary field in crystal Report.</u>

In Crystal Reports, a **Summary Field** is used to perform calculations on a specific set of data within the report, such as totals, averages, counts, and other aggregate functions. Summary fields are helpful when you need to display summarized or aggregated data, especially in reports like financial statements, sales summaries, and performance reports.

Types of Summary Fields:

1. **Sum**: Adds the values in a specified field.
2. **Count**: Counts the number of records in a field.
3. **Average**: Computes the average value of the field.
4. **Maximum**: Returns the maximum value in a field.
5. **Minimum**: Returns the minimum value in a field.
6. **Distinct Count**: Counts the number of unique values in a field.

(D) Attempt Any One : **5**

1. <u>Explain types of reports in details</u>.

Standard Report

Description:

This is the most commonly used report type. It displays data in a simple list or tabular format.

Use:

Ideal for listing data such as employee records, product inventories, or sales transactions.

Features:

- Easy to design and format
- Supports grouping and sorting
- Can include summary fields, charts, and formulas

2. Cross-Tab Report

Description:

Displays summarized data in a matrix format (rows and columns), similar to pivot tables in Excel.

Use:

Perfect for comparing related data, like monthly sales across different regions.

Features:

- Shows aggregated data (Sum, Avg, Count, etc.)
- Rows and columns are dynamic based on selected fields

3. Subreport

Description:

A report embedded inside another report, used to display related or additional data.

Use:

Useful when combining data from different sources or showing child records.

Features:

- Can be linked to main report data
- Operates independently or in sync with the main report

4. Drill-Down Report

Description:

Allows users to click on summary data to view detailed underlying data.

Use:

Used for interactive reporting where summary data needs exploration.

Features:

- Users can expand/collapse sections
- Reduces clutter by hiding detailed data until needed

5. Parameterized Report

Description:

Accepts user inputs (parameters) to filter or customize report data.

Use:

Ideal for creating flexible reports where users select a date range, category, or department.

Features:

- Enhances reusability
- Reduces the need for multiple versions of a report

6. Chart Report

Description:

Uses visual charts (bar, pie, line, etc.) to represent data graphically.

Use:

Great for visualizing trends, comparisons, and distributions.

Features:

- Supports different chart types
- Can be combined with other report types

7. Mailing Label Report

Description:

Generates formatted labels from data fields like name and address.

Use:

Used for creating address labels, product tags, etc.

Features:

- Designed for label sheets (e.g., Avery formats)
- Supports formatting and spacing

8. Form Report

Description:

Designed to look like a form or document (e.g., invoice, bill, purchase order).

Use:

Common in business operations for document generation.

Features:

- Static layout with embedded fields
- Often includes logos, headers, footers

2.<u>Write a steps to create crystal Report.</u>

Step 1: Create a New Project

- Open **Visual Studio**.
- Create a new **Windows Forms Application** project.

Step 2: Add Crystal Report

- Right-click on the project → **Add → New Item**.
- Select **Crystal Report** from the list.
- Name the report (e.g., EmployeeReport.rpt) and click **Add**.

Step 3: Choose a Report Creation Method

- Choose one of the options:
  - **Using the Report Wizard**
  - **As a Blank Report**
- Click **OK**.

Step 4: Connect to a Data Source

- Choose your database connection (e.g., SQL Server, OLE DB).
- Select or create a connection string.
- Choose the table or stored procedure you want to use.

Step 5: Design the Report

- Drag and drop fields from the **Field Explorer** onto the report sections:
  - **Page Header**: For titles and column headers
  - **Details**: For actual data rows
  - **Page/Footer**: For footers, summaries, or page numbers

Step 6: Insert Summary, Group, or Charts (Optional)

- Use **Insert** → Summary, Group, or Chart to enhance your report.

Step 7: Add CrystalReportViewer to Form

- Open the Form (e.g., Form1).
- Drag **CrystalReportViewer** from the toolbox onto the form.

Step 8: Load the Report in Code

```
using CrystalDecisions.CrystalReports.Engine;
ReportDocument rpt = new ReportDocument();
rpt.Load("EmployeeReport.rpt"); // or use full path
crystalReportViewer1.ReportSource = rpt;
```

Step 9: Run the Application

- Press **F5** to run the project.
- The report will be displayed in the CrystalReportViewer.